

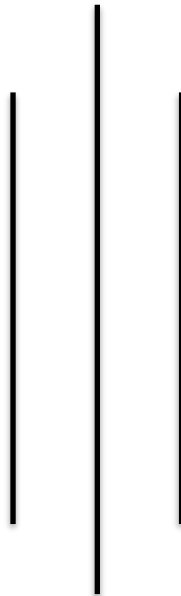
A Case Study

On

Student Hostel Management System

Submitted partial fulfillment of the requirement of

Software Engineering (BIT 354CO)



Submitted By:

Sushant Thapa (Roll No: 40)

Submitted To:

Ananta Bhadra Lamichhane

KANTIPUR CITY COLLEGE

Putalisadak, Kathmandu

August, 2026

Acknowledgement

The developer extends sincere gratitude to the academic and technical supervisors for their valuable guidance, support, and encouragement throughout the preparation of this case study. Their vast knowledge and constructive suggestions helped establish a robust understanding of software engineering concepts, allowing for the practical application of theoretical frameworks. The opportunity to explore real-world software engineering practices and enhance analytical and problem-solving skills is deeply appreciated. The continuous guidance provided by the advisory board has played an instrumental role in making this software engineering case study a success.

Table of Contents

Chapter 1: Introduction	1
1.1 Background	1
1.2 Problem Statement	1
1.3 Introduction	2
1.3.1 Project Objectives.....	2
1.3.2 System Scope.....	2
1.3.3 Project Scope	3
1.3.4 Nature of Anticipated Software	3
1.3.5 Essential Characteristics of Anticipated Software.....	3
1.4 Project Packaging.....	4
1.4.1 Work Breakdown Structure (WBS).....	4
1.4.2 Deliverables and Milestones.....	4
1.4.3 Acceptance Criteria	5
1.4.4 Project Approach	6
1.5 Project Resourcing	7
1.5.1 Quality Requirements	7
1.5.2 Process Requirements.....	7
1.5.3 Method Requirements.....	8
1.5.4 Resource Requirements	9
Chapter 2: Planning and Management	10
2.1 Project Risks.....	10
2.1.1 Identified Risk	10
2.1.2 Mitigation Risk	11
2.1.3 Contingency Plan.....	11
2.1.4 Risk Monitoring Plan.....	12
2.2 Project Estimation and Schedule.....	12
2.2.1 Estimation Method Used	12
2.2.2 Project Estimation.....	12
2.2.3 Project Schedule	13

2.3 Configuration Management Plan	13
2.3.1 Change Management Plan	13
2.3.2 Version Management Plan.....	14
2.3.3 Release Management Plan.....	14
Chapter 3: System Engineering	15
3.1 Requirements.....	15
3.1.1 Domain Requirements	15
3.1.2 Functional Requirements	15
3.1.3 Non-Functional Requirements.....	16
3.1.4 Requirement Management Process	16
3.2 Design.....	17
3.2.1 System Architecture	17
3.2.2 High Level Designs	18
3.2.3 Data Structure Design.....	19
3.3 Testing.....	22
3.3.1 Test Justification.....	22
3.3.2 Test Plan	22
3.3.3 Test Design	23
Chapter 4: Monitoring & Control	24
4.1 Monitoring & Control	24
4.1.1 Time Monitoring.....	24
4.1.2 Cost Monitoring.....	24
4.1.3 Control Measures.....	25
4.2 Quality Assurance	25
4.2.1 QMS Selection.....	25
4.2.2 Quality Factors	25
4.2.3 Metrics for Process and Product.....	26
4.2.4 Quality Monitoring	26
Annex 1: Requirement Models	27

Chapter 1: Introduction

1.1 Background

The operational complexity of managing educational accommodations has surged significantly in the contemporary digital era. Historically, hostel administration relied upon manual registers, scattered spreadsheets, and error-prone paper trails to track student occupancy, fee collection, and inventory usage. These traditional methods are highly susceptible to data redundancy, delayed financial reconciliation, and miscommunication between hostel authorities and students. When student records, room allocations, and mess bills are tracked manually, retrieving the historical data of a previous student or calculating real-time room vacancies becomes a labor-intensive task.

A dedicated web-based Student Hostel Management System (SHMS) presents an optimal, digitized solution to these systemic failures. By centralizing databases, automating routine tasks, and enforcing stringent data validation, a software-driven approach optimizes resource utilization and radically improves the residential experience for students. The transition to a computerized infrastructure minimizes human error, secures valuable administrative data for extended periods, and enables instantaneous data retrieval, fundamentally transforming the institutional workflow.

1.2 Problem Statement

Managing student hostels through manual or fragmented digital systems remains a significant bottleneck for educational institutions. The limitations in existing paradigms contribute to poor financial tracking, suboptimal room allocation, and degraded maintenance responses. The core issues necessitating this software intervention are identified through the following specific parameters:

Specific: Manual ledgers and disconnected spreadsheets fail to consistently record real-time room occupancies, student check-in/out timestamps, and accurately categorized fee payments, leading to highly disorganized administrative data and financial leakage.

Measurable: Institutions utilizing manual processes report data entry redundancies, missing entries, and billing calculation errors in a vast majority of their end-of-month audits, requiring extensive manual reconciliation.

Achievable: A scalable, centralized web application utilizing a relational database backend can be systematically constructed within an estimated seventeen-week timeframe to fully automate these workflows.

Relevant: Accurate occupancy tracking and automated billing algorithms directly improve financial liquidity, regulatory compliance, operational efficiency, and overall student satisfaction.

Time-bound: The proposed software system must be capable of generating instant capacity reports, issuing fee invoices, and processing algorithmic allocations on demand, completely eliminating the lag inherently found in manual accounting processes.

1.3 Introduction

1.3.1 Project Objectives

The fundamental objectives guiding the development of the Student Hostel Management System dictate the architectural and functional priorities of the software engineering lifecycle.

Priority	Objective
1	Develop a centralized, web-based platform for managing student profiles, room allocations, and administrative records to replace existing paper-based systems.
2	Automate financial operations, including dynamic fee structure generation, invoice processing, penalty calculations, and payment tracking.
3	Establish an interactive portal for students to submit maintenance requests, view mess menus, and track their occupancy status and dues.
4	Ensure highly secure, normalized relational database storage that eliminates redundant entries, maintains referential integrity, and provides restricted data access based on user roles.

1.3.2 System Scope

The operational limits defining the capabilities of the software are strictly delineated to manage expectations and ensure technical feasibility.

Priority	System Boundary
1	Administrators hold exclusive privileges to dynamically add, edit, and disable room categories, hostel blocks, and physical assets.

2	The system executes algorithmic room allocation based on predefined constraints, managing capacity limits, room types, and waitlist reservations.
3	The system mandates secure user authentication, tracking daily attendance, visitor logs, and check-in/check-out timestamps across distinct roles (Admin, Warden, Student).
4	All persistent data is stored on a centralized relational database server, providing a single source of truth for the entire institution.

1.3.3 Project Scope

The project deliverables and software engineering lifecycle phases are defined strictly to prevent feature bloat and ensure timely delivery.

Priority	Project Extent
1	Comprehensive requirements elicitation, feasibility analysis, and preparation of the Software Requirements Specification (SRS) detailing all functional behaviors.
2	Advanced design of the system architecture, graphical user interfaces, and deeply normalized Entity-Relationship (ER) schemas extending up to the Third Normal Form.
3	Implementation of interconnected core modules using a modern programming framework, establishing the application logic for room management and financial tracking.
4	Rigorous software testing, detailed technical documentation, and final deployment of the system to the institution's localized or cloud servers.

1.3.4 Nature of Anticipated Software

The nature of the Anticipated Software is:

- Centralized multi-user web application
- Interactive transaction-based system
- Custom-built software for academic hostel management
- Relational database storage with concurrent access support

1.3.5 Essential Characteristics of Anticipated Software

The essential characteristics of anticipated software are:

- Usability: Simple and intuitive graphical user interface for non-technical staff

- Reliability: Accurate calculations and consistent data storage
- Maintainability: Modular and well-structured source code
- Portability: Accessible via standard web browsers on different operating systems
- Efficiency: Fast response for allocation, billing, and report generation
- Security: Role-based access control to protect sensitive data

1.4 Project Packaging

1.4.1 Work Breakdown Structure (WBS)

To manage the inherent complexities of a multi-module web application, the project is divided into hierarchical tasks, allowing for structured tracking and execution. The initial phase focuses entirely on requirement engineering, involving deep stakeholder communication to extract domain rules, followed by feasibility studies and the drafting of the formal specification document. The subsequent phase encompasses architectural and relational database design, heavily relying on entity-relationship modeling and normalization techniques. Following the design approval, the construction phase initiates, divided into backend algorithmic development for room allocation and billing engines, and frontend user interface integration. The final phase shifts to rigorous quality assurance, encompassing unit testing, integration testing, and comprehensive system testing prior to production deployment.

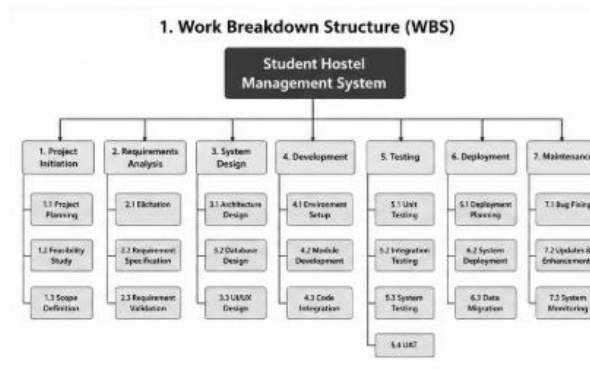


Fig 1.1: Work Breakdown Structure (WBS) – Student Hostel Management System

1.4.2 Deliverables and Milestones

Based on predictive software engineering mathematical models, the project timeline is structured around critical milestones that track the delivery of functional components.

Sequence	Deliverable	Milestone Completion
1	Validated Project Topic, Feasibility Study, and Initial Project Plan	Week 2
2	Finalized Software Requirements Specification (SRS) and Data Flow Models	Week 4
3	System Design Document encompassing UML diagrams and Normalized Database Schema	Week 7
4	Working Prototype featuring User Authentication and Core Room Allocation Logic	Week 11
5	Complete Integrated Application featuring Billing, Mess Management, and Reporting	Week 15
6	Final Documentation, User Acceptance Testing, and Project Submission	Week 17

1.4.3 Acceptance Criteria

The product will be considered complete and ready for institutional handover only when the following rigorous criteria are unambiguously met.

S.N.	Acceptance Criteria
1	The application initializes and runs successfully on the target web server environment without requiring manual dependency interventions.
2	Administrators can successfully execute complete CRUD operations on student profiles, room configurations, and staff assignments.
3	The automated room allocation logic correctly assigns students to available beds without ever exceeding the mathematically defined maximum room capacities.
4	The billing engine accurately calculates and generates end-of-month invoices, correctly aggregating base room rent, mess subscriptions, and applicable disciplinary penalties.
5	Role-based access control (RBAC) securely prevents standard students from viewing global financial ledgers, peer records, or administrative dashboards.
6	The system prevents temporal scheduling conflicts by validating proposed dates against existing check-in and check-out logs before finalizing bookings.
7	All relational database transactions pass strict referential integrity checks.
8	Input validation protocols actively prevent invalid or malicious data from being processed or saved.

9	The application passes all structured unit testing, integration testing, and User Acceptance Testing (UAT) phases.
10	Comprehensive user manuals and technical architectural documentation are finalized and formally approved by the supervisory board.

1.4.4 Project Approach

The Incremental Process Model was selected for the development of the Student Hostel Management System because of the following reasons:

1. Modular Development

The application consists of separate modules such as User Management, Room Allocation, Billing, and Maintenance Tracking. Each module can be developed independently.

2. Early Working Software

A functional version of the application is available after each increment, allowing progress to be evaluated throughout development.

3. Continuous Testing

Every completed module is tested before integrating it with the next module, helping to detect and fix defects early.

4. Easy Maintenance

Changes or improvements can be made to a specific module without affecting the entire application.

5. Reduced Development Risk

Developing the project in smaller increments minimizes the risk of major errors and makes debugging easier.

6. Better Project Management

The project can be planned, monitored, and completed in phases, making development more organized and manageable.

7. Suitable for the Project

Since the Student Hostel Management System is a web-based application with clearly defined and independent functionalities, the Incremental Process Model provides an efficient and systematic development approach.

8. Fits the project scenario

With a focused development team, a 17-week academic timeline, and clearly separable modules with stable requirements, incremental delivery provides a low-risk and well-managed path to completion.

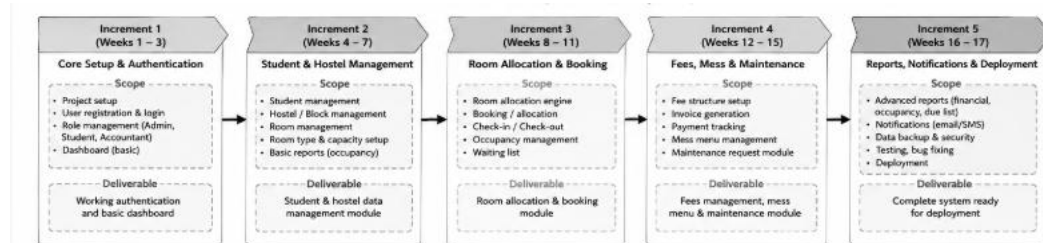


Fig 1.2: Incremental Module

1.5 Project Resourcing

1.5.1 Quality Requirements

The quality requirements the project aims to fulfil are:

- Correctness of calculations and data storage
- Reliability during normal use
- High usability through a simple interface
- Maintainable source code
- Completeness of core features
- Security of student and financial data

1.5.2 Process Requirements

The quality requirements are mapped to the Incremental Process Model as follows:

S.N.	Quality Requirement	Mapped Activity in Incremental Process	How Quality is Ensured
1	Correctness	Testing phase of each increment	Every increment is tested individually before integration
2	Reliability	Continuous validation across increments	Each new increment is validated against previously delivered increments

3	Usability	Design phase of each increment	UI is reviewed and refined based on feedback from early working versions
4	Maintainability	Modular development structure	Each increment is built as an independent module
5	Completeness	Incremental delivery sequence	Each increment adds a clearly defined feature set

1.5.3 Method Requirements

Throughout the development of the Student Hostel Management System, a set of key framework and umbrella activities shall be followed to ensure compliant and systematic process adoption.

The framework activities, representing the core process steps carried out within every increment, are as follows:

- **Communication:** Requirements are clarified and validated with the project supervisor before the start of each increment.
- **Planning:** Tasks for each increment are scheduled and organized to maintain a structured development timeline.
- **Modeling:** The architecture, user interface, and database design relevant to each increment are prepared prior to implementation.
- **Construction:** Each module is coded and unit-tested to verify correctness before integration.
- **Deployment:** The completed and tested increment is integrated into the overall system and delivered as a working version.

In addition, the following umbrella activities are carried out continuously:

- **Project tracking and monitoring**
- **Risk management**
- **Software quality assurance**
- **Software configuration management**
- **Documentation management**

1.5.4 Resource Requirements

The successful development of the Student Hostel Management System requires the following technological, platform, environment, and human resources.

Technological Resources

- Programming Language / Framework: PHP with Laravel or Python with Django
- Frontend: HTML5, CSS3, JavaScript
- Database: MySQL or PostgreSQL
- Development Tool: VS Code / XAMPP / Docker
- Version Control: Git

Platform and Environment

- Operating System: Windows / Linux / macOS for development; Linux server for production
- Hardware: Standard desktop/laptop; institutional server or cloud instance for deployment

Human Resources

- Project Developer: 1 (Student)
- Project Supervisor: 1
- End Users for feedback: Wardens and 2–3 students

Chapter 2: Planning and Management

2.1 Project Risks

2.1.1 Identified Risk

Software engineering projects inherently carry significant threats that can compromise schedules, budgets, and the quality of the final deliverable. Proactive risk management requires identifying these factors early and quantifying their Probability and Impact on a standardized scale of 1 to 5. The Risk Priority Number (RPN) is calculated as $\text{Probability} \times \text{Impact}$. Risks are subsequently sorted by their RPN to determine which vulnerabilities require the most urgent attention.

Risk ID	Risk Description	Category	Prob.	Impact	RPN	Priority
R1	Unanticipated scope creep driven by evolving administrative requirements	Product	4	4	16	High
R2	Relational database design flaws causing calculation failures or data corruption	Technical	2	5	10	High
R3	Integration failures between room allocation and billing engine	Technical	3	4	12	High
R4	Underestimation of effort for RBAC and security cryptography	Schedule	3	3	9	Medium
R5	Suboptimal UI design leading to administrative friction and user rejection	Product	2	4	8	Medium
R6	Hardware failure or repository compromise causing source code loss	Support	1	5	5	Low
R7	Underestimated testing timelines leading to schedule slippage	Schedule	3	4	12	High
R8	Gaps in input validation allowing invalid or malicious entries	Cost	3	2	6	Low

2.1.2 Mitigation Risk

Risk ID	Mitigation Strategy
R1	Elicit exhaustive requirements initially and enforce a strict Change Request protocol. Lock core functionality early.
R2	Adhere rigidly to 3NF and BCNF during initial database design; use junction tables for many-to-many relationships.
R3	Develop a mock API bridge between allocation and billing modules early to validate data payloads.
R4	Allocate specific buffer time in the schedule dedicated solely to security and authorization logic.
R5	Conduct UI/UX wireframe reviews and prototype sessions with administrative staff before production frontend code.
R6	Maintain rigorous Git commit protocols, push to remote repository daily, and keep automated local backups.
R7	Integrate testing phases directly into each increment of the WBS rather than deferring QA to the end.
R8	Implement strict server-side input validation during construction of each module.

2.1.3 Contingency Plan

Risk ID	Contingency Action (if risk occurs)
R1	Halt new feature development and reduce scope to MVP; defer enhancements to future patches.
R2	Revert to last stable schema migration, execute data cleansing, and re-run affected test suites.
R3	Temporarily decouple modules; process financial transactions via offline batch script until bridge is repaired.
R4	Request schedule extensions or reallocate hours from non-critical reporting modules to security core.
R5	Switch to a simplified utilitarian layout featuring basic data tables to save development time.
R6	Restore latest Git commit from remote repository and rebuild local environment from secured source.
R7	Prioritize testing core acceptance criteria first; document non-critical UI bugs as known issues.
R8	Deploy immediate hotfix for validation logic and permanently add a regression test for the failed input.

2.1.4 Risk Monitoring Plan

Risk ID	Monitoring Activity	Frequency
R1	Compare planned vs actual progress against WBS; review pending Change Requests	Weekly
R2	Verify database schema integrity and execute sample-data constraint checks	Every session
R3	Monitor successful transmission rates of simulated data payloads between modules	Weekly
R4	Track time spent per security module against WBS estimate	Weekly
R5	Solicit informal feedback on interface usability from peers	Bi-Weekly
R6	Audit Git commit history and verify integrity of remote backups	Weekly
R7	Review ratio of completed versus planned test cases	Weekly / after increment
R8	Run hostile validation test cases against every new input field	After module completion

2.2 Project Estimation and Schedule

2.2.1 Estimation Method Used

The Basic Constructive Cost Model (COCOMO 81) operating in Organic Mode was selected for effort and schedule estimation. The software operates within a familiar problem domain (hostel management), features a small focused team, and entails relatively flexible operational constraints. Using empirical algorithmic models rather than arbitrary guesswork prevents severe budget overruns and establishes a realistic baseline for project tracking.

2.2.2 Project Estimation

Based on the modular complexity of the SHMS—which encompasses extensive CRUD operations, complex algorithmic room allocations, dynamic financial billing engines, and analytical reporting—the estimated source code volume is projected at 1.6 KLOC.

Basic COCOMO 81 Formula (Organic Mode):

$$\text{Effort (E)} = 2.4 \times (\text{KLOC})^{1.05}$$

$$\text{Development Time (T)} = 2.5 \times (\text{E})^{0.38}$$

$$E = 2.4 \times (1.6)^{1.05} \approx 3.9 \text{ Person-Months}$$

$$T = 2.5 \times (3.9)^{0.38} \approx 4.2 \text{ Months (approximately 17 weeks)}$$

Suggested Team Size $\approx$ 1 dedicated full-time developer

Assuming a regional development cost rate of NPR 30,000 per month:

$$\text{Total Cost} = 3.9 \times 30,000 \approx \text{NPR } 117,000$$

2.2.3 Project Schedule

The project schedule is mapped using a Gantt Chart framework. This visual timeline distributes all major activities of the Student Hostel Management System across the 17-week projection. The initial weeks are front-loaded with requirement gathering and database modeling. The middle segment dictates parallel development tracks for backend logic and frontend interfaces. The final weeks are strictly reserved for integration testing, user acceptance evaluations, and final documentation.

Gantt Chart - Student Hostel Management System (17 Weeks)

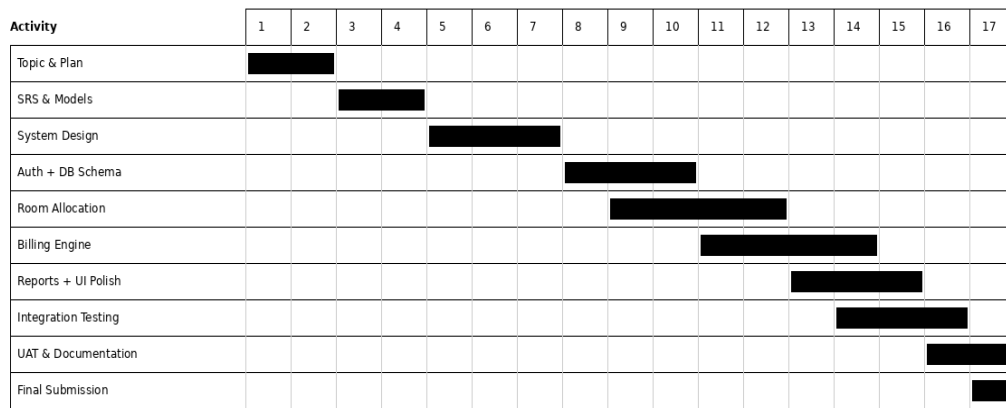


Fig 2.1: Project Schedule Gantt Chart (17-week timeline)

Fig 2.1: Project Schedule Gantt Chart (17-week timeline)

2.3 Configuration Management Plan

2.3.1 Change Management Plan

Any proposed change is first documented in a Change Request (CR) Log, recording the change description, reason, requester, priority, and expected impact on the schedule. Each change is reviewed against the current project scope and remaining schedule before a decision is made. Approved changes are incorporated into the next development increment. Rejected changes are recorded with reasons for future reference.

2.3.2 Version Management Plan

Git is used as the version control tool. Semantic versioning is followed (for example: v0.1, v0.5, v0.8, v1.0). Each important milestone is tagged so that stable versions can be restored if needed.

2.3.3 Release Management Plan

Release 1: Core authentication and database schema

Release 2: Room allocation and category management

Release 3: Billing engine and reporting

Final Release (v1.0): Complete system after UAT and documentation approval

Chapter 3: System Engineering

3.1 Requirements

3.1.1 Domain Requirements

The system must inherently understand and enforce the unyielding domain rules of physical hostel operations:

- Support recording of detailed student demographic and academic profiles.
- A room cannot be allocated to a number of students exceeding its defined physical capacity.
- Financial generation must accommodate dynamic variables including prorated room rent, fluctuating mess charges, and ad-hoc disciplinary penalties.
- No financial transaction, invoice, or check-in record may be hard-deleted; soft deletion must be used to preserve audit trails.
- All sensitive financial data and personal demographics must be secured using robust encryption standards.

3.1.2 Functional Requirements

Priority	Functional Requirement	Description
High	FR-01: User Authentication	Secure login with strict RBAC restricting interface and database privileges by role.
High	FR-02: Room Management	Administrators manage hostel rooms: type, capacity, base pricing, and physical location.
High	FR-03: Student Allocation	Complex room allocation handling assignment logic, waiting lists, and historical tracking.
High	FR-04: Billing and Payments	Autonomous fee management, itemized monthly invoices, and payment processing.
High	FR-05: Check-in/Check-out	Digital check-in/out with exact timestamps managing real-time room availability.
Medium	FR-06: Maintenance Requests	Students submit maintenance tickets; admins track, assign, and resolve them.

Medium	FR-07: Inventory Tracking	Track furniture and appliances tied to specific rooms or blocks.
Medium	FR-08: Reporting Analytics	Generate reports on occupancy rates, outstanding ledgers, and maintenance speeds.

3.1.3 Non-Functional Requirements

S.N.	Category	Non-Functional Requirement
1	Usability	Administrative interface must be intuitive; staff can allocate rooms and generate invoices with minimal training.
2	Reliability	Financial calculations must be flawless; transaction locking prevents data corruption during concurrent operations.
3	Performance	Complex relational joins must execute and render within 2–3 seconds under normal server load.
4	Maintainability	Source code must adhere to MVC; modules remain highly cohesive but loosely coupled.
5	Scalability	Database architecture must support growth from hundreds to thousands of student profiles.
6	Security	Cryptographic standards (e.g., bcrypt) protect credentials; defend against SQL injection and XSS.

3.1.4 Requirement Management Process

The engineering team employed a multi-phase requirement management process consisting of elicitation, analysis, modeling, and specification. Elicitation was conducted through communication with institutional stakeholders and questionnaires with wardens. Analysis categorized requirements into domain, functional, and non-functional parameters, resolving conflicts against the 17-week timeline by prioritizing core acceptance criteria. Modeling produced Use Case diagrams and Data Flow Diagrams. Specification documented validated

requirements in the formal SRS, which was then frozen as the binding blueprint for design and construction.

3.2 Design

3.2.1 System Architecture

The system architecture of the Student Hostel Management System follows a layered architecture style. This architecture divides the application into multiple layers, where each layer has a specific responsibility and only communicates with the layer directly below it.

The layered architecture of the proposed system is shown in the figure below:

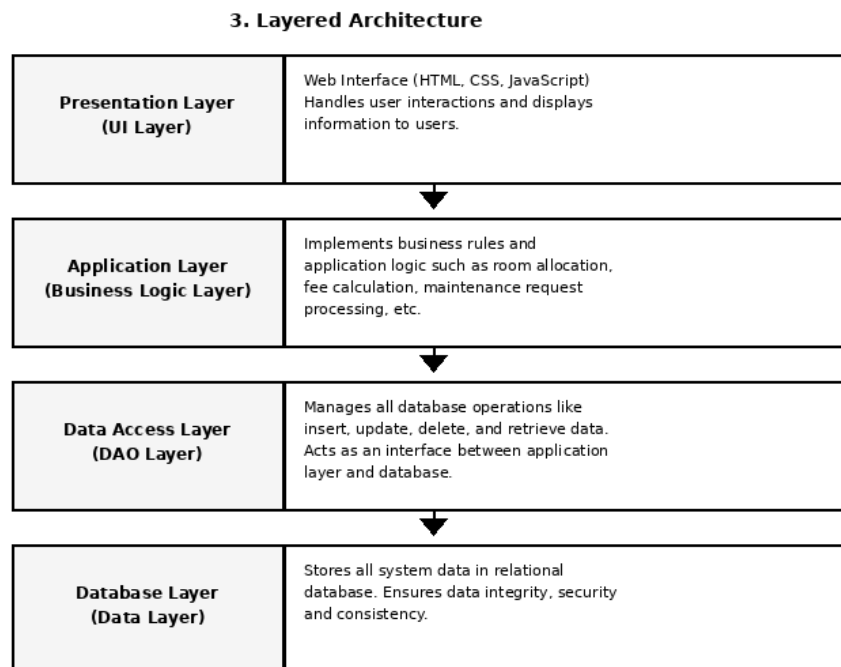


Fig 3.1: Layered Architecture

Description of Layers:

- **Presentation Layer (UI Layer):** Provides the web interface using HTML, CSS, and JavaScript. It handles user interactions and displays information such as dashboards, forms, and reports to Admin, Warden, and Student users.
- **Application Layer (Business Logic Layer):** Implements the main business rules and application logic of the system such as room allocation, fee

calculation, invoice generation, maintenance request processing, and role-based authorization.

- **Data Access Layer (DAO Layer):** Manages all database operations like insert, update, delete, and retrieve. It acts as an interface between the application layer and the database, keeping query logic separate from business logic.
- **Database Layer (Data Layer):** Stores all system data in a relational database (MySQL/PostgreSQL). It ensures data integrity, security, and consistency for students, rooms, allocations, invoices, and payments.
- This layered design ensures separation of concerns, easy maintenance, and better modularity of the Student Hostel Management System application.

3.2.2 High Level Designs

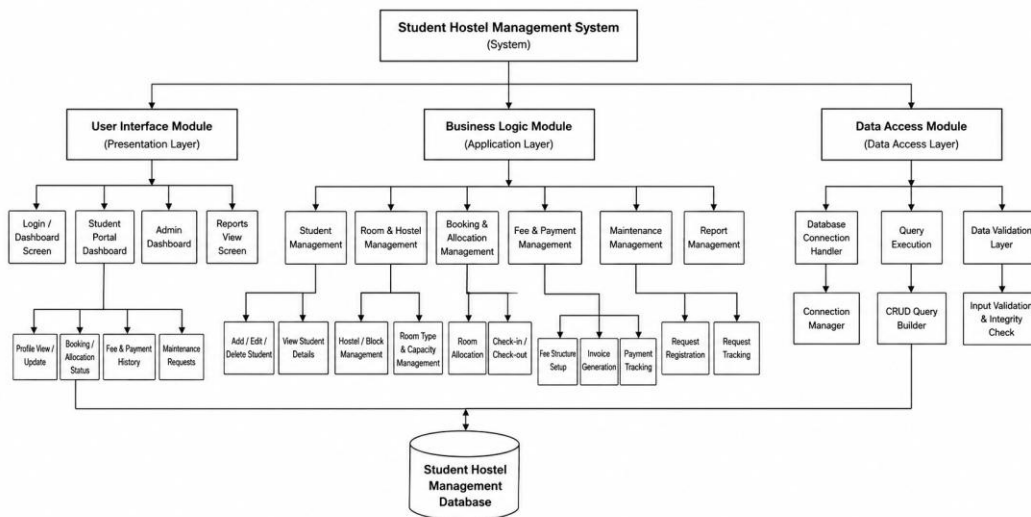


Fig 3.2: High Level Diagram

The High-Level Diagram of the Student Hostel Management System shows the overall structure of the system and how its major modules are connected. At the top is the Student Hostel Management System, which is divided into three main modules: the User Interface Module, Business Logic Module, and Data Access Module. The User Interface Module provides screens such as the login/dashboard, student portal, admin dashboard, reports, booking/allocation status, fee history, and maintenance requests. The Business Logic Module handles the core operations of the system, including student management, hostel and room management, booking

and allocation, fee and payment management, maintenance, and report management.

The Data Access Module manages communication between the application and the database. It includes the database connection handler, query execution, and data validation components, which perform database connections, CRUD operations, and input/integrity checks. All information is ultimately stored in the Student Hostel Management Database, including student, room, booking, payment, and maintenance-related records. Thus, the diagram provides a high-level view of how users interact with the system, how business operations are processed, and how data is stored and retrieved.

3.2.3 Data Structure Design

The SHMS database is highly normalized up to the Third Normal Form (3NF) and Boyce-Codd Normal Form (BCNF), eliminating transitive dependencies and redundant data. Core entities are detailed below.

Entity: room_types

Field Name	Data Type	Key Constraints	Description
id	INTEGER	Primary Key (PK)	Unique identifier for the room category
name	VARCHAR(50)	NOT NULL	Category name (e.g., Single, Twin Air-Con)
capacity	INTEGER	NOT NULL	Maximum number of beds allowed
base_price	DECIMAL(10,2)	NOT NULL	Standard monthly rent applied for billing

Entity: rooms

Field Name	Data Type	Key Constraints	Description
id	INTEGER	Primary Key (PK)	Unique identifier for the physical room

hostel_block_id	INTEGER	Foreign Key (FK)	Links the room to a specific building/block
room_type_id	INTEGER	Foreign Key (FK)	References room_types.id for price and capacity
room_number	VARCHAR(10)	NOT NULL	Alphanumeric physical identifier (e.g., A-101)
status	VARCHAR(20)	NOT NULL	Real-time state (e.g., 'Available', 'Maintenance')

Entity: students

Field Name	Data Type	Key Constraints	Description
student_id	INTEGER	Primary Key (PK)	Unique database identifier
enrollment_no	VARCHAR(50)	UNIQUE, NOT NULL	Official university-issued identification
full_name	VARCHAR(100)	NOT NULL	Legal name of the student
contact_no	VARCHAR(15)	NOT NULL	Primary communication channel
admission_date	DATE	NOT NULL	Date of institutional enrollment

Entity: hostel_allocations (Junction Table)

Field Name	Data Type	Key Constraints	Description
id	INTEGER	Primary Key (PK)	Unique allocation identifier

student_id	INTEGER	Foreign Key (FK)	References students.student_id
room_id	INTEGER	Foreign Key (FK)	References rooms.id
start_date	DATE	NOT NULL	Timestamp of official check-in
end_date	DATE	NULLABLE	Timestamp of check-out or anticipated departure
status	VARCHAR(20)	NOT NULL	Lifecycle state ('Active', 'Completed', 'Cancelled')

Entity: invoices

Field Name	Data Type	Key Constraints	Description
id	INTEGER	Primary Key (PK)	Unique billing document identifier
allocation_id	INTEGER	Foreign Key (FK)	Links invoice to a room stay
issue_date	DATE	NOT NULL	Timestamp of invoice generation
subtotal	DECIMAL(10,2)	NOT NULL	Base charges before tax or penalties
tax	DECIMAL(10,2)	NOT NULL	Government or institutional levies
total_amount	DECIMAL(10,2)	NOT NULL	Final calculated sum due
status	VARCHAR(20)	NOT NULL	Payment state ('Paid', 'Unpaid', 'Overdue')

Entity: payments

Field Name	Data Type	Key Constraints	Description
id	INTEGER	Primary Key (PK)	Unique transaction identifier
invoice_id	INTEGER	Foreign Key (FK)	References invoices.id
payment_date	TIMESTAMP	NOT NULL	Exact chronological time of the transaction
amount	DECIMAL(10,2)	NOT NULL	Monetary value successfully transferred
txn_reference	VARCHAR(100)	UNIQUE	External banking or payment gateway reference

3.3 Testing**3.3.1 Test Justification**

Complex relational databases and web applications require rigorous testing protocols to prevent catastrophic failures during peak institutional enrollment periods. A defect in room allocation logic could assign multiple students to a single bed. A logical flaw in the financial engine could generate invalid invoices. Comprehensive testing proves system reliability, validates business logic constraints, uncovers security vulnerabilities, and ensures data persistence under server duress.

3.3.2 Test Plan

The testing methodology integrates both black-box and white-box techniques. Black-box testing validates the system against user workflows defined in the SRS. White-box testing scrutinizes internal logic, SQL query efficiency, encryption strength, and data validation pathways. The process progresses hierarchically: unit testing, integration testing, and system testing. Pass criteria require that all core

CRUD operations function without data loss, all financial arithmetic resolves with zero margin of error, and the system actively rejects hostile inputs.

3.3.3 Test Design

Test Module: Room Allocation Logic

Test ID	Scenario to Test	Execution Steps	Expected Outcome
AL-01	Standard Allocation	Assign unallocated student to available room with capacity	Allocation entry committed; room availability decreases by 1
AL-02	Over-capacity Rejection	Allocate student to room already at full capacity	Transaction rejected; explicit capacity error returned
AL-03	Concurrent Conflict	Two admins allocate the final bed at the same time	First succeeds; second fails gracefully with alert

Test Module: Billing and Financial Integrity

Test ID	Scenario to Test	Execution Steps	Expected Outcome
FIN-01	Invoice Generation	Trigger end-of-month billing for active student	Invoice populates with correct rent + mess charges
FIN-02	Partial Payment	Input payment less than total invoice amount	Payment recorded; invoice status updates to Partial
FIN-03	Data Persistence	Terminate DB mid multi-table financial insert	Rollback preserves previous stable state; no orphans
FIN-04	Hostile Data Injection	Input negative value or text into payment amount	Server-side validation rejects payload before query

Chapter 4: Monitoring & Control

4.1 Monitoring & Control

4.1.1 Time Monitoring

Time monitoring of the Student Hostel Management System project is executed utilizing a Gantt Chart framework. This visual timeline tracks all major developmental phases across the projected 17-week schedule. By overlaying actual progress against the planned baseline on a weekly basis, project managers can instantly visualize whether specific increments are adhering to the critical path. Regular milestone reviews ensure that testing phases are not compressed due to preceding developmental delays..

Gantt Chart - Student Hostel Management System (17 Weeks)

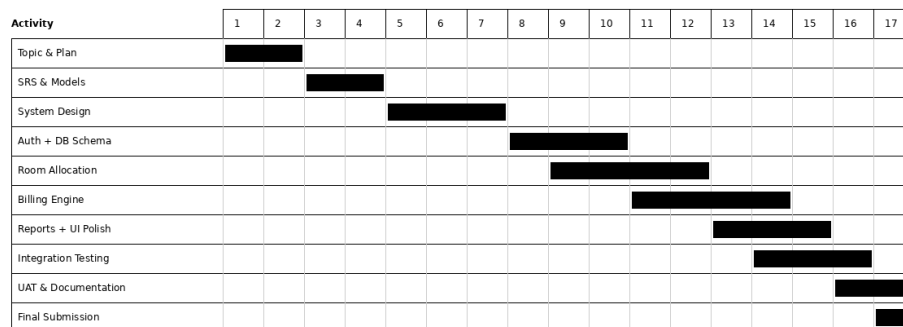


Fig 2.1: Project Schedule Gantt Chart (17-week timeline)

Fig: 4.1: Gantt Chart

4.1.2 Cost Monitoring

To ensure the project adheres to COCOMO 81 estimations, the methodology incorporates Earned Value Management (EVM). EVM quantifies objective progress against consumed time and resources.

Planned Value (PV): Budgeted cost of work scheduled.

Earned Value (EV): Quantified value of work actually delivered and verified.

Actual Cost (AC): Labor hours and capital consumed to achieve that Earned Value.

Schedule Performance Index (SPI) = EV / PV

Cost Performance Index (CPI) = EV / AC

An SPI of 1.0 means the project is perfectly on schedule. A CPI of 1.0 means

financial efficiency matches the plan. By tracking SPI and CPI weekly, managers gain empirical insight into deviations.

4.1.3 Control Measures

Type of Deviation	Diagnostic Cause	Control Measures
Time Delay (SPI < 1)	Modules requiring more debugging time than estimated	Enforce time-boxing; defer non-critical visual enhancements; focus on stabilizing core logic
Cost Overrun (CPI < 1)	Expanded labor hours due to misunderstood scope	Halt coding; emergency SRS review to freeze scope; restrict development to MVP features
Severe Deviation (Both)	Technology incompatibility or data model failure	Trigger contingency plans: simplify architecture, use third-party components, negotiate timeline extensions

4.2 Quality Assurance

4.2.1 QMS Selection

The Quality Management System selected as the base reference is ISO 9001, applied in a lightweight form suitable for academic projects. It provides a clear framework for defining processes, reviewing work products, and controlling quality. Its plan-do-check-act principles align with the Incremental development approach used in this project.

4.2.2 Quality Factors

Preference Order	Quality Factor	Reason for Preference
1	Correctness	Financial calculations and allocation logic must be mathematically accurate
2	Reliability	System must not lose data or fail during peak admission operations

3	Security	Sensitive student and financial data require strong protection
4	Usability	Wardens and staff may lack advanced technical literacy
5	Maintainability	Modular MVC structure enables future team updates
6	Performance	Reports and queries must respond within 2–3 seconds

4.2.3 Metrics for Process and Product

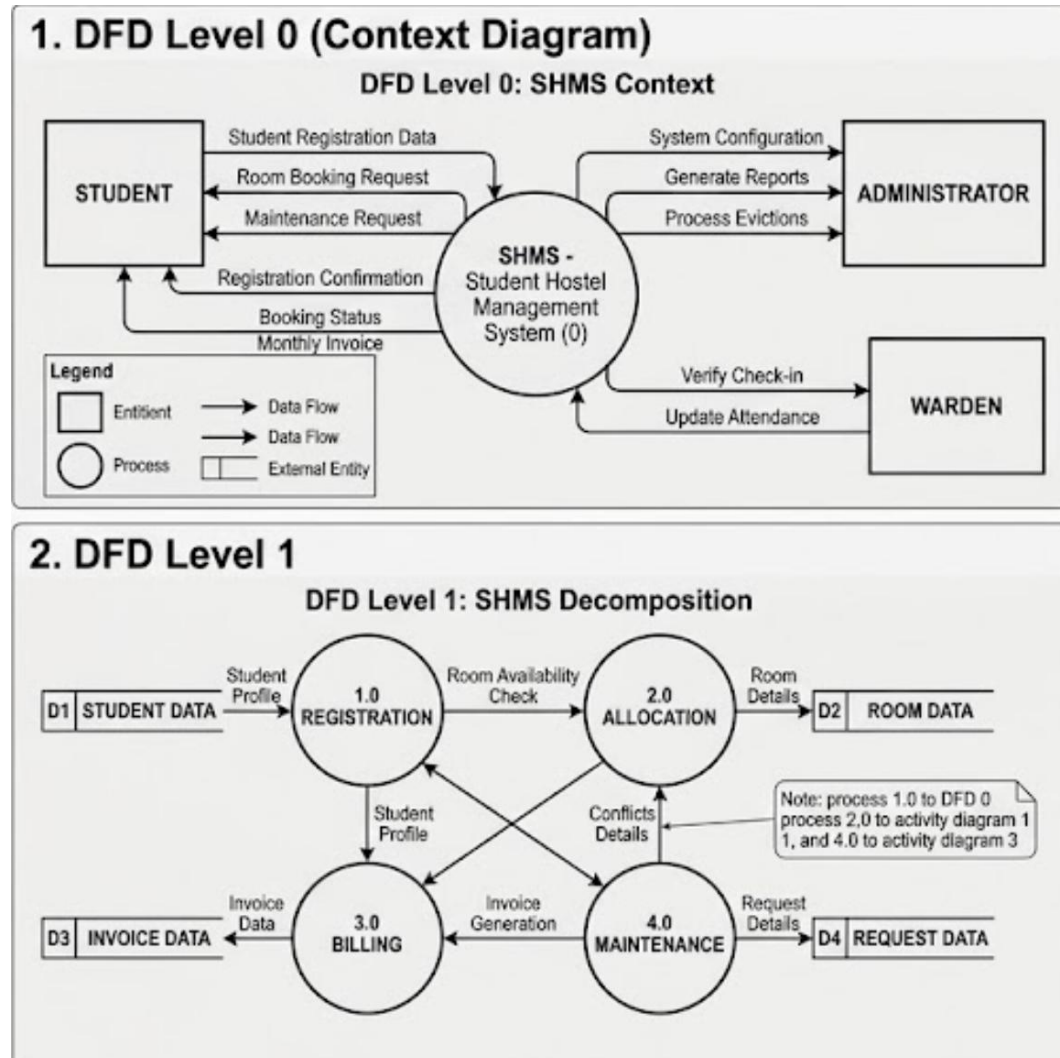
Metric	Description	Target / Result
Schedule Variance	Difference between planned and actual milestone dates	SV = 0 (on plan)
Effort Variance	Planned effort vs actual effort	Within COCOMO estimate
Defect Removal Efficiency	$DRE = E / (E + D)$	Target $\geq 95\%$
Requirements Coverage	% of functional requirements implemented and tested	100%
Test Cases Passed	Percentage of planned test cases passed	100% critical cases

4.2.4 Quality Monitoring

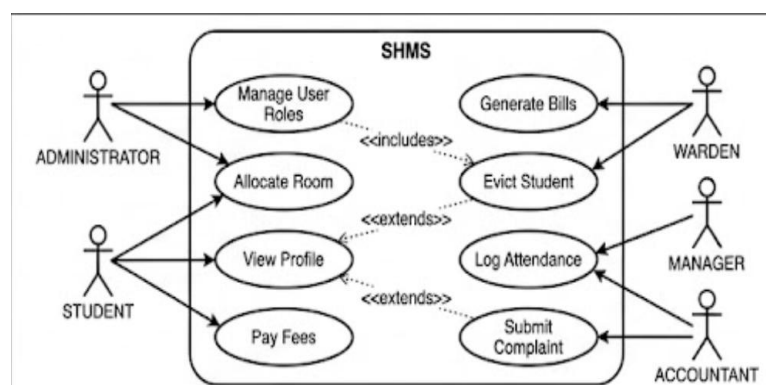
Quality is monitored through weekly progress reviews against the Gantt Chart, code and design reviews after each increment, unit testing after each module, integration testing after combining modules, and system/acceptance testing before final submission. Templates used include Test Case Tables, Defect Logs, Change Request Logs, and Test Summary Reports. Quality is measured against Acceptance Criteria, Functional/Non-Functional Requirements, the Test Plan, planned schedule estimates, and selected Quality Factors. Defects found during testing are fixed before moving to the next increment. Requirements are frozen after the SRS phase to avoid late changes.

Annex 1: Requirement Models

1. Data Flow Diagram



2. Use Case Diagram



3. Activity Diagram

